

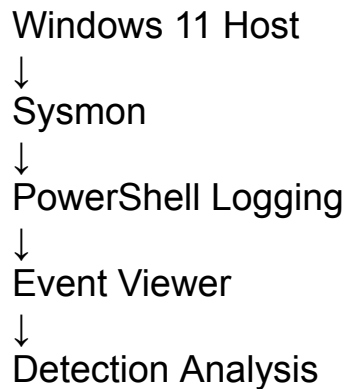
Detection Engineering Home Lab using Sysmon and PowerShell Logging

Project Overview

This project was created to understand endpoint telemetry, Windows event logging, and detection engineering concepts using Sysmon and native PowerShell logging.

The objective was to generate controlled activities, observe resulting telemetry, investigate events, and understand visibility gaps within logging configurations.

Lab Architecture



Tools Used:

- Sysmon
- PowerShell
- Event Viewer
- Windows 11

Project Statistics:

Detections Created: 7

Sysmon Event IDs Investigated: 1, 3, 11

PowerShell Event IDs Investigated: 4103, 4104

Telemetry Sources:

- Sysmon
- PowerShell Operational Logs

Key Concepts:

- Process Creation
- File Creation
- Network Connections
- Parent-Child Processes
- PowerShell Logging
- Telemetry Validation
- Visibility Gap Analysis

Detection Engineering Home Lab using Sysmon and PowerShell Logging

Project Overview

This project was created to understand endpoint telemetry, Windows event logging, and detection engineering concepts using Sysmon and native PowerShell logging.

The objective was to generate controlled activities, observe resulting telemetry, investigate events, and understand visibility gaps within logging configurations.

Lab Architecture

Windows 11 Host
↓
Sysmon
↓
PowerShell Logging
↓
Event Viewer
↓
Detection Analysis

Tools Used:

- Sysmon
- PowerShell
- Event Viewer
- Windows 11

Detection Use Cases

Detection 001 - Local Account Creation

Objective:

Observe process creation telemetry generated when creating a local user account.

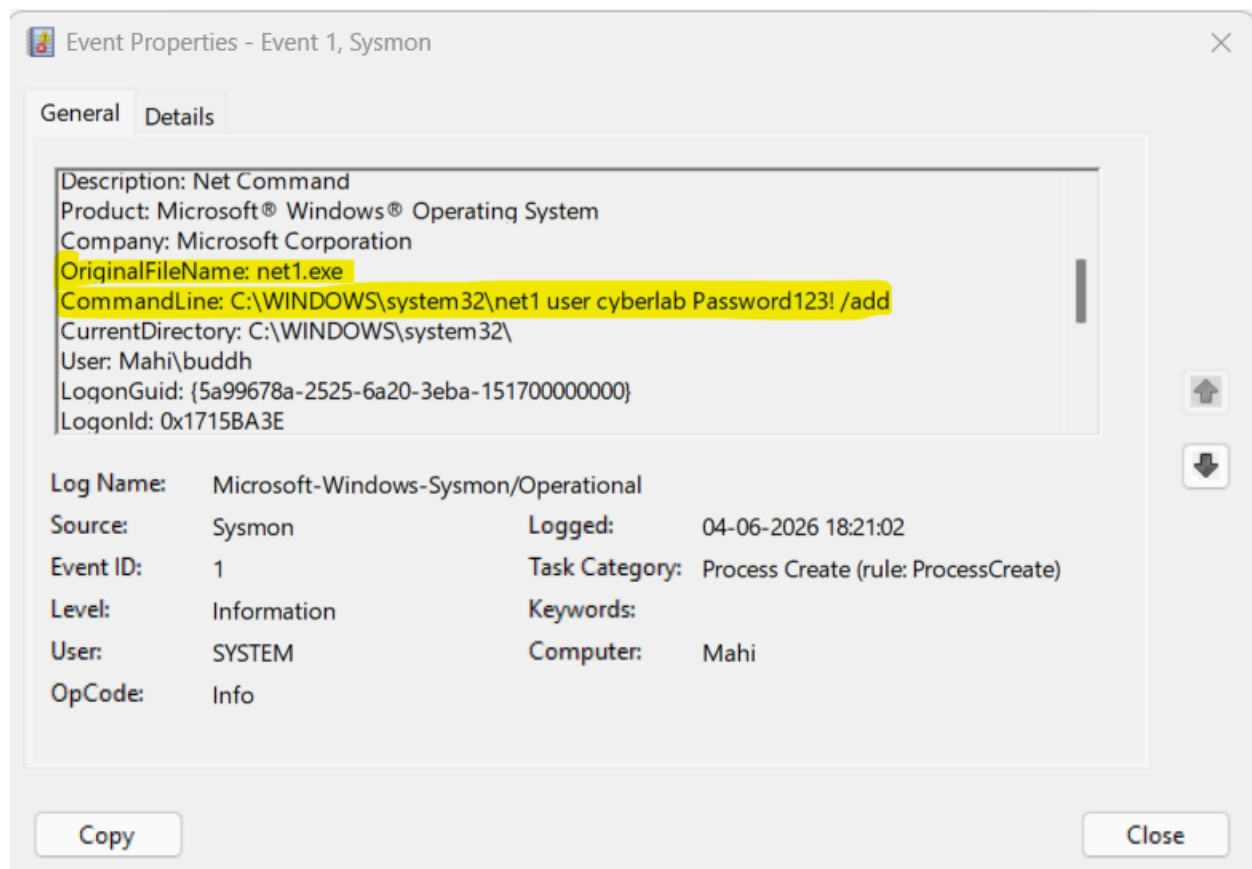
Event IDs:

- Sysmon Event ID 1

Key Findings:

- Process creation events record command-line arguments.
- Parent-child relationships provide execution context.

Evidence:



Detection 002 - PowerShell File Creation

Objective:

Investigate file creation activity generated by PowerShell.

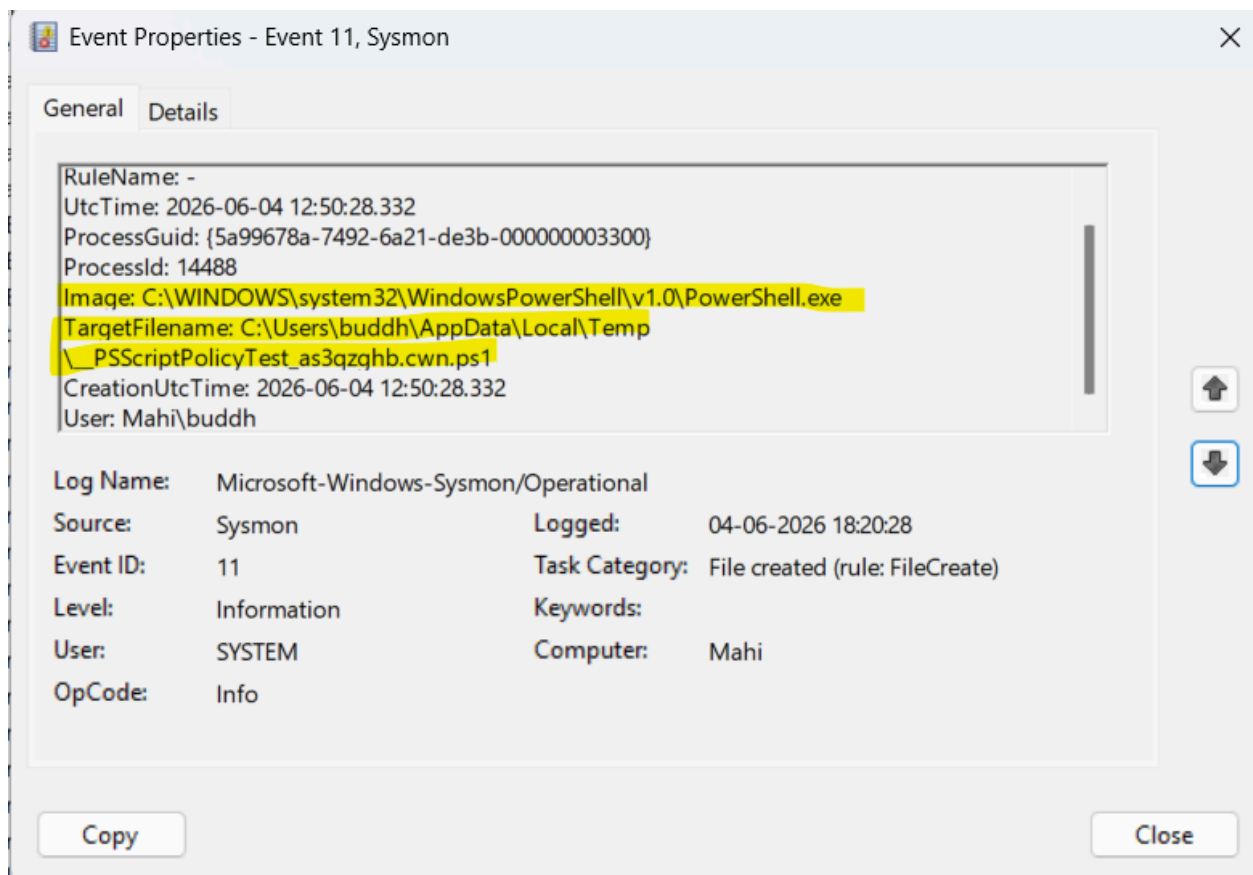
Event IDs:

- Sysmon Event ID 11

Key Findings:

- File creation events contain process context.
- Not all file creations indicate malicious activity.

Evidence:



Detection 003 - PowerShell Network Connection

Objective:

Observe outbound network connections initiated by PowerShell.

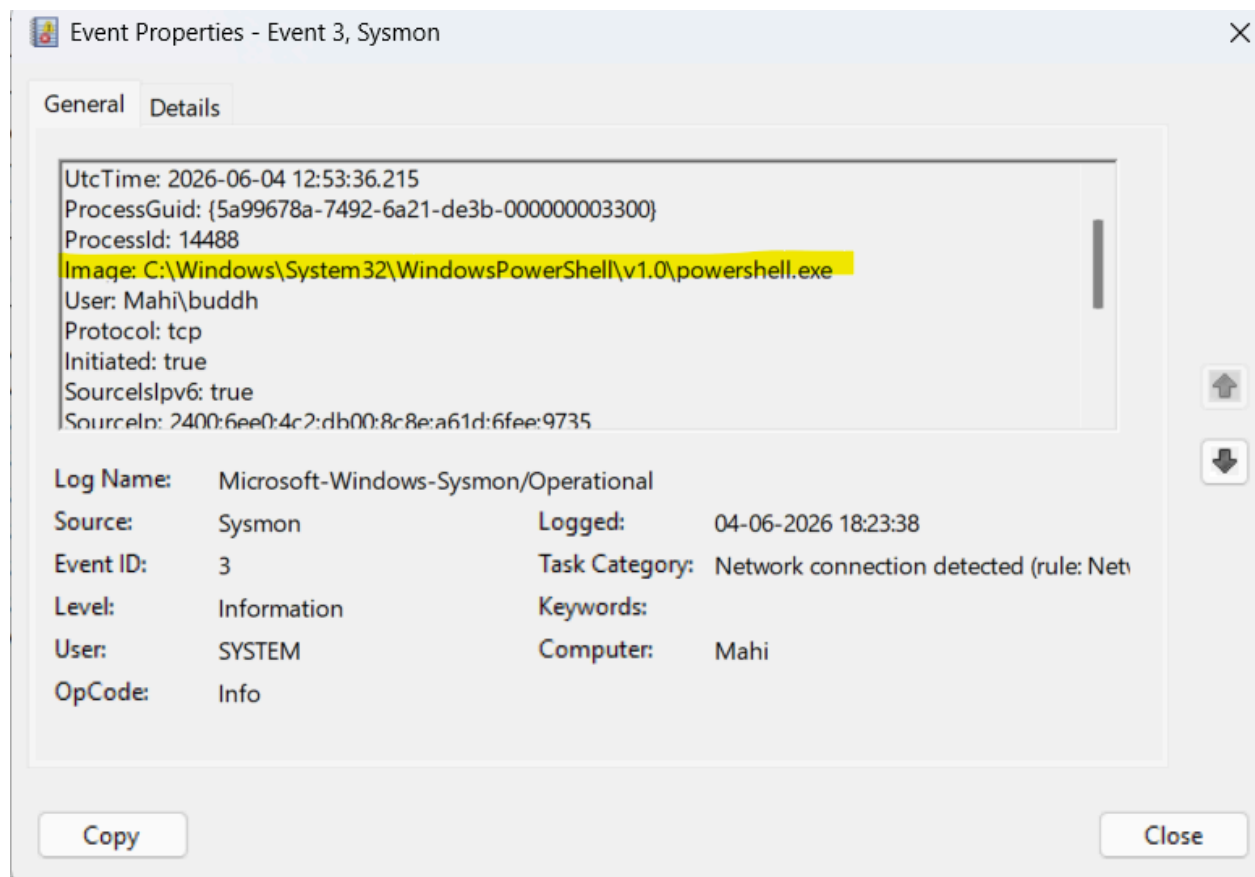
Event IDs:

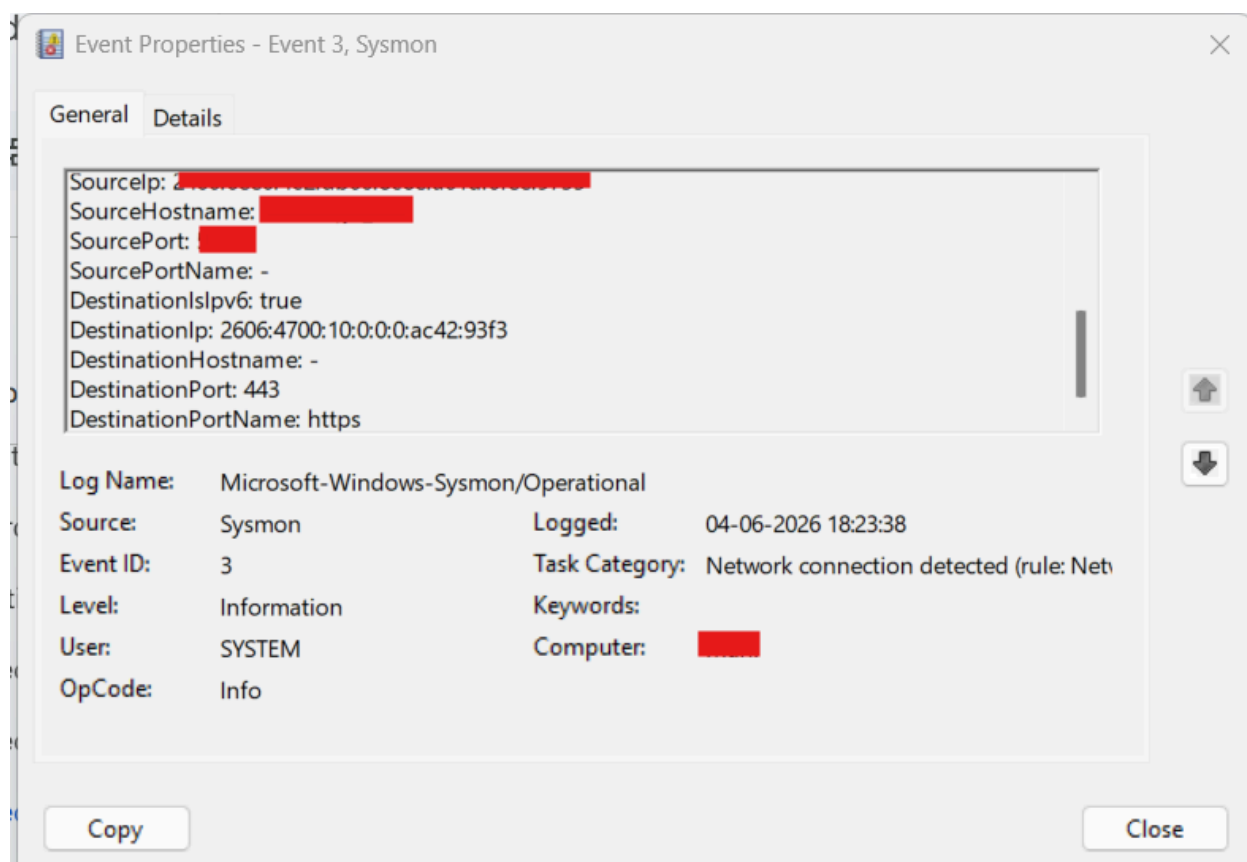
- Sysmon Event ID 3

Key Findings:

- Network telemetry identifies destination IPs and ports.
- Context is required before classifying activity as suspicious.

Evidence:





Detection 004 - PowerShell Child Process Creation

Objective:

Observe parent-child process relationships when PowerShell launches another process.

Event IDs:

- Sysmon Event ID 1 (Process Creation)

Key Findings:

- Parent-child relationships provide valuable execution context.
- ParentImage identifies which process launched a child process.
- Process chains help analysts understand how activity originated.

Evidence:

Image:

Notepad.exe

ParentImage:
PowerShell.exe

CommandLine:
"C:\WINDOWS\system32\notepad.exe"

ParentCommandLine:
"C:\WINDOWS\system32\WindowsPowerShell\v1.0\PowerShell.exe"

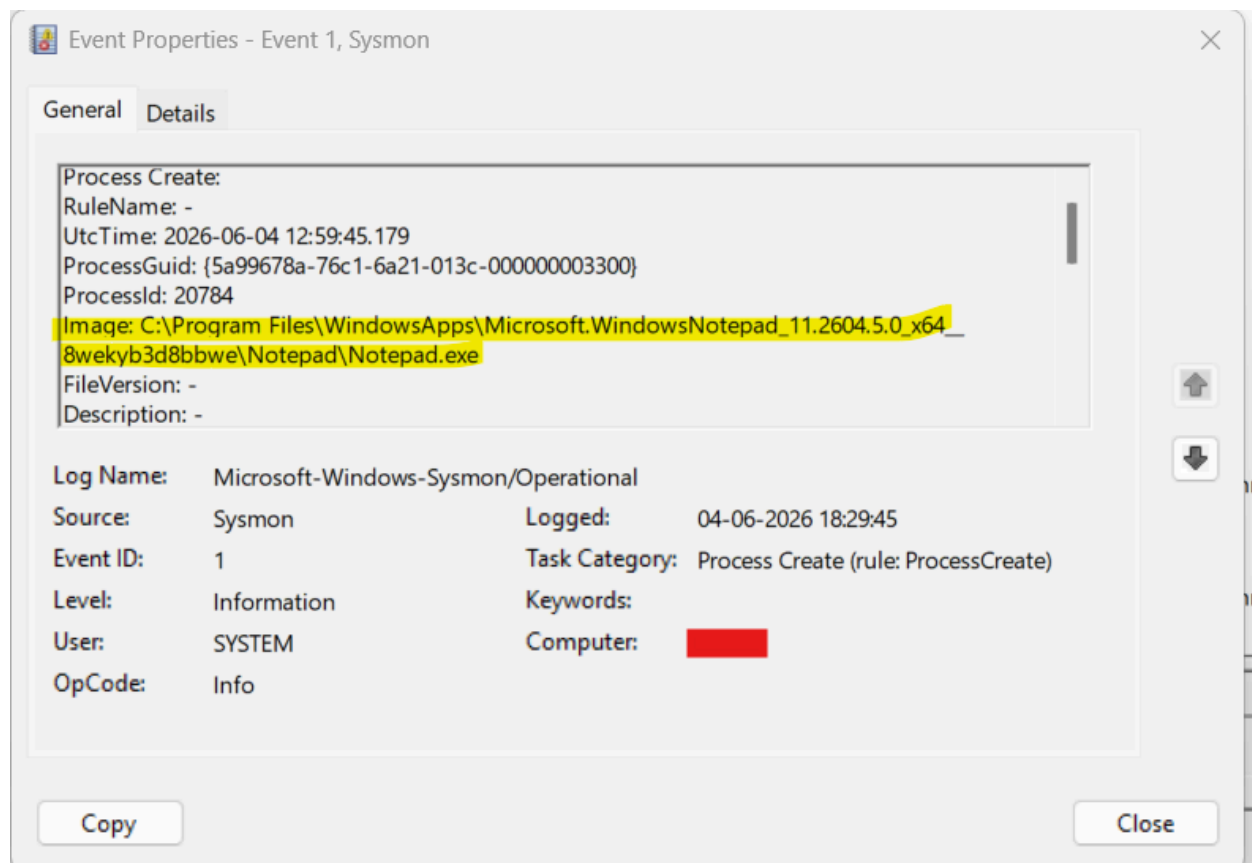
Assessment:

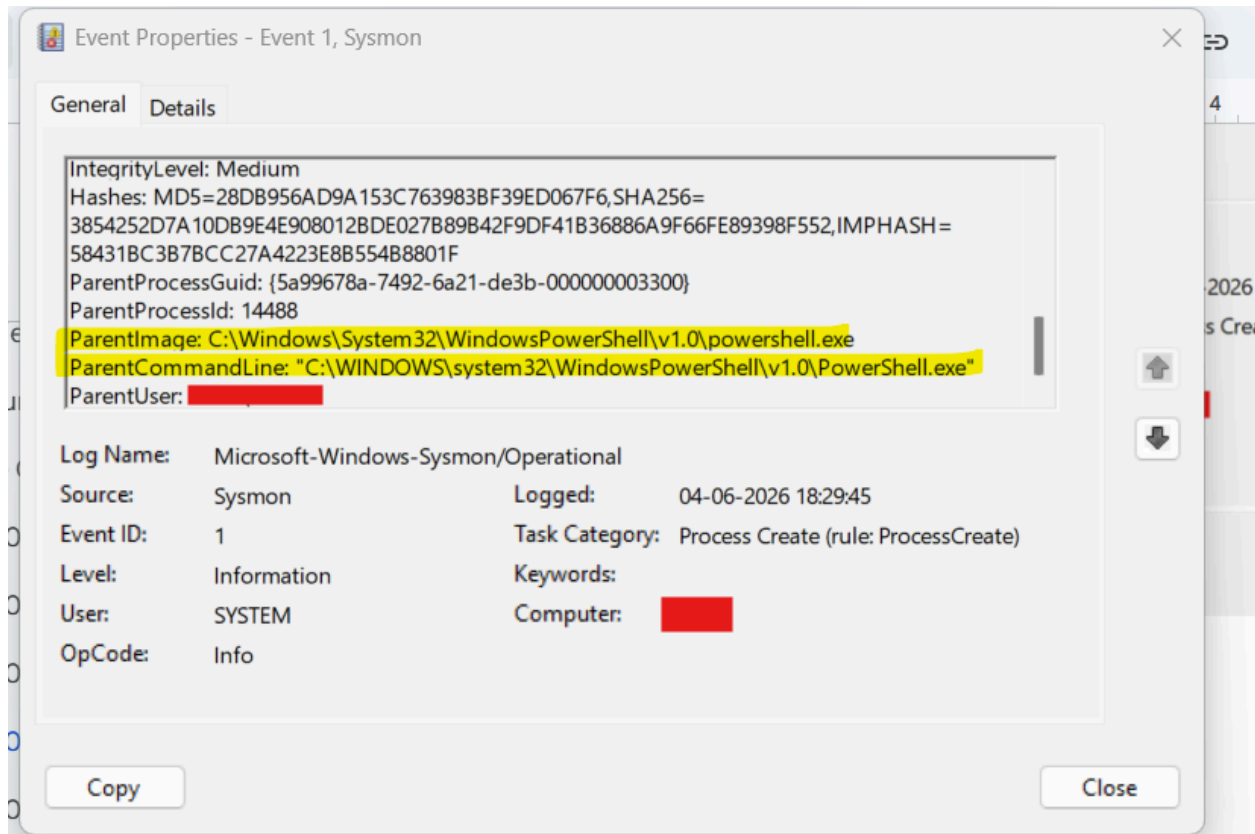
PowerShell successfully launched Notepad as a child process. This behavior was expected and generated during a controlled lab exercise.

Why It Matters:

Attackers frequently use PowerShell to launch additional processes. Monitoring parent-child process relationships helps identify suspicious execution chains and attacker activity.

Evidence:





Detection 005 - PowerShell Command Execution Logging

Objective:

Enable and validate PowerShell Script Block Logging to capture commands executed within PowerShell sessions.

Event IDs:

- PowerShell Event ID 4104

Key Findings:

- Script Block Logging captures PowerShell command content.
- Visibility extends beyond process creation telemetry.
- Commands executed within existing PowerShell sessions can be monitored and investigated.

Evidence:

Command Executed:
Get-Process

Event ID:
4104

User:
[Redacted]

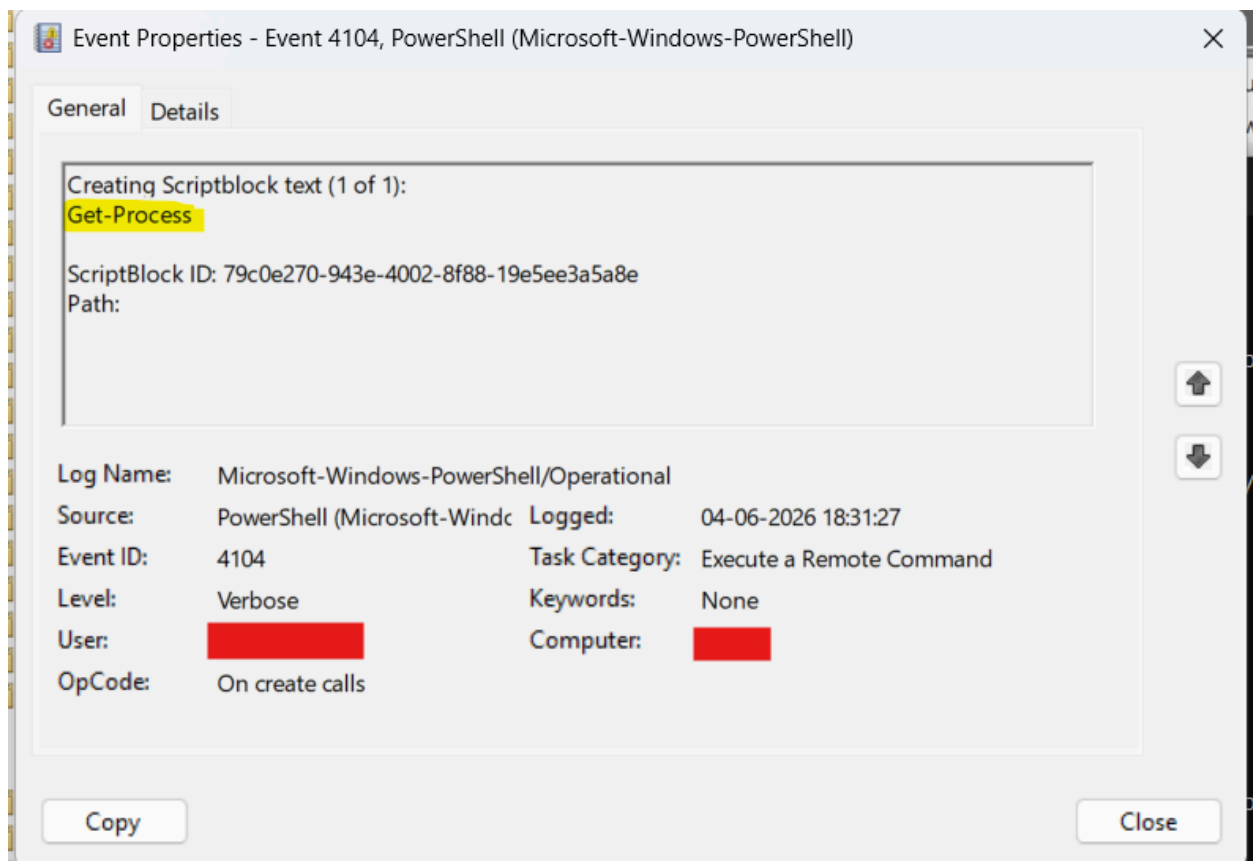
Assessment:

PowerShell Script Block Logging successfully recorded command execution during controlled testing.

Why It Matters:

Process creation logs show that PowerShell executed, but Script Block Logging reveals the actual commands that were run. This provides greater visibility into administrative and potentially malicious PowerShell activity.

Evidence:



Detection 006 - PowerShell Download Activity

Objective:

Generate PowerShell download activity and observe command execution telemetry.

Event IDs:

- PowerShell Event ID 4104

Activity Performed:

Invoke-WebRequest <https://example.com> -OutFile "\$env\example.html"

Key Findings:

- Script Block Logging successfully captured the PowerShell download command.
- Download activity can be reconstructed from PowerShell telemetry.
- Command content provides valuable context during investigations.

Evidence:

Command:

Invoke-WebRequest <https://example.com> -OutFile "\$env\example.html"

Event ID:

4104

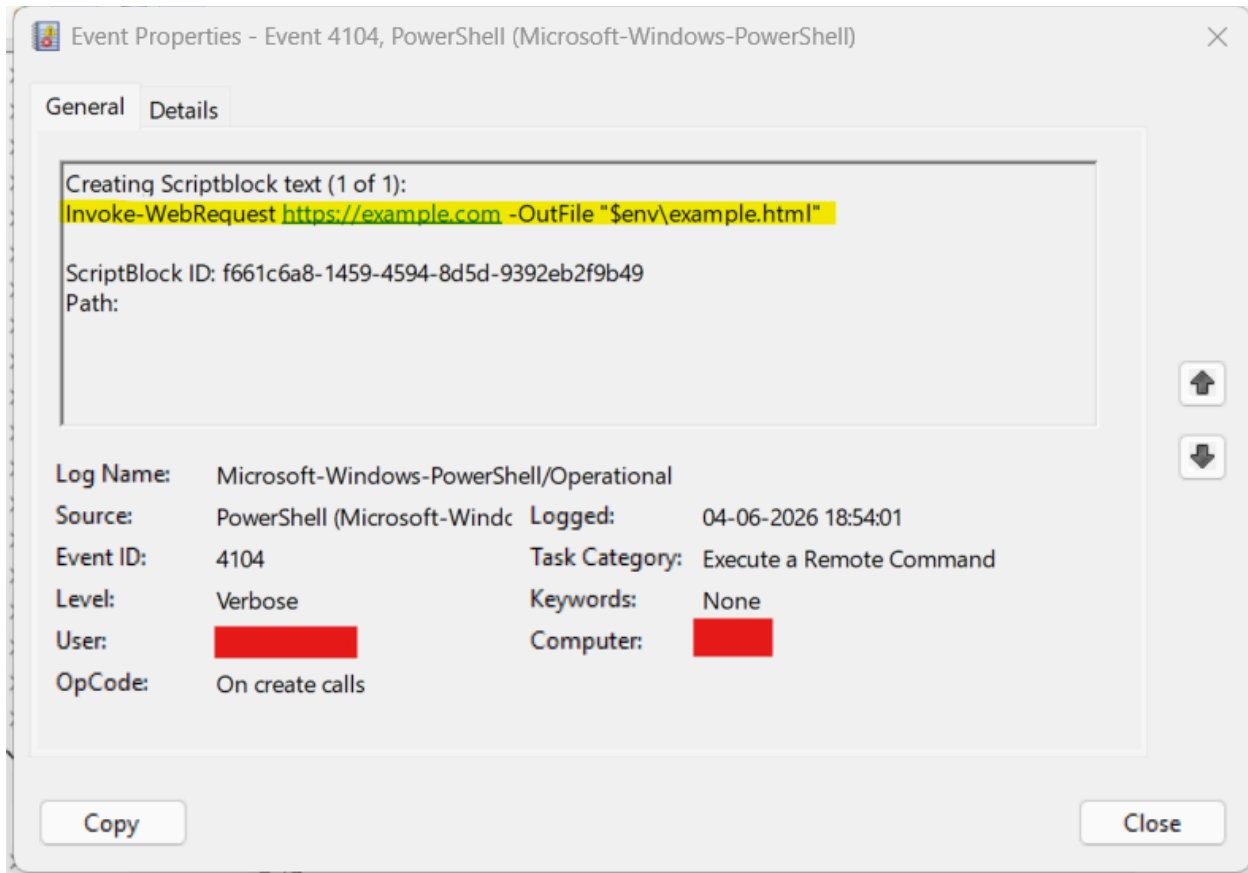
Assessment:

The command successfully downloaded content from example.com and saved it locally during controlled testing.

Why It Matters:

PowerShell is frequently used to download scripts, payloads, and tools. Visibility into download commands helps analysts identify potentially malicious activity and understand attacker behavior.

Evidence:



Detection 007 - Telemetry Gap Analysis

Objective:

Validate telemetry coverage by comparing expected logging behavior against observed activity.

Related Activity:

Invoke-WebRequest <https://example.com> -OutFile "\$env:TEMP\example.html"

Expected Telemetry:

- PowerShell Event ID 4104
- Sysmon Event ID 11 (File Creation)

Observed Telemetry:

- Event ID 4104 observed
- Downloaded file successfully created
- No corresponding Sysmon Event ID 11 observed

Investigation:

The downloaded file was verified to exist within the Temp directory. Review of the Sysmon configuration revealed that FileCreate monitoring used include-based filtering and did not monitor the Temp directory where the file was created.

Assessment:

The activity executed successfully, but expected file creation telemetry was absent due to Sysmon configuration scope.

Root Cause:

Sysmon FileCreate monitoring relied on include-based rules that excluded the file location used during testing.

Why It Matters:

Detection coverage depends on telemetry quality and configuration. Missing telemetry can create visibility gaps that affect detection effectiveness and investigative visibility.

Evidence:

Screenshot of Sysmon FileCreate configuration

```
<!--SYSMON EVENT ID 11 : FILE CREATED [FileCreate]-->
<!--EVENT 11: "File created"-->
<!--NOTE: Other filesystem "minifilters" can make it appear to Sysmon that some files are being written twice. This is not a Sysmon issue, per Mark Russinovich.-->
<!--NOTE: You may not see files detected by antivirus. Other filesystem minifilters, like antivirus, can act before Sysmon receives the alert a file was written.-->

<!--DATA: UtcTime, ProcessGuid, ProcessId, Image, TargetFilename, CreationUtcTime-->
<RuleGroup name="" groupRelation="or">
  <FileCreate onmatch="include">
    <TargetFilename name="T1023" condition="contains">\Start Menu\TargetFilename> <!--Windows: Startup links and shortcut modification
[ https://attack.mitre.org/wiki/Technique/T1023 ] -->
    <TargetFilename name="T1165" condition="contains">\Startup\</TargetFilename> <!--Microsoft:Changes to user's auto-launched files and shortcuts-->
    <TargetFilename name="OutlookAttachment" condition="contains">\Content.Outlook\</TargetFilename> <!--Microsoft:Outlook: attachments-->
    <TargetFilename name="Downloads" condition="contains">\Downloads\</TargetFilename> <!--Downloaded files. Does not include "Run" files in IE-->
    <TargetFilename condition="end with">.application\TargetFilename> <!--Microsoft:clickonce: [ https://blog.netspi.com/all-you-need-is-one-a-clickonce-love-
```

Screenshot verifying downloaded file exists

```
PS C:\WINDOWS\system32> dir $env:TEMP\example.html

Directory: C:\WINDOWS\TEMP

Mode                LastWriteTime         Length Name
----                -
-a----           04-06-2026   17:45           528 example.html
```

Telemetry Gap Analysis

Scenario:

PowerShell executed the following command:

Invoke-WebRequest <https://example.com> -OutFile "\$env:TEMP\example.html"

Expected Telemetry:

- PowerShell Script Block Logging (4104)
- Sysmon Network Connection (3)
- Sysmon File Creation (11)

Observed Telemetry:

- Event ID 4104 observed
- File successfully created
- No corresponding Event ID 11 observed

Investigation:

Review of the Sysmon configuration revealed that FileCreate monitoring used an include-based rule set and did not include the Temp directory where the file was created.

Root Cause:

The Sysmon configuration intentionally limited file creation logging to selected paths and file types.

Key Lesson:

Successful activity does not guarantee successful visibility. Detection coverage depends on telemetry configuration and monitoring scope.

Key Learnings

- Process creation telemetry provides execution context.
- Parent-child process relationships are critical for investigations.
- PowerShell Script Block Logging provides visibility beyond process creation logs.
- Different telemetry sources answer different investigative questions.
- Logging configurations directly affect detection coverage.
- Visibility gaps can be identified through controlled testing and validation.

Conclusion

This project demonstrated how endpoint telemetry can be used to investigate system activity, build detections, and identify visibility gaps. Through controlled testing and analysis, multiple Windows and PowerShell telemetry sources were explored to better understand detection engineering principles.